

SQL DDL скрипт

```
1 CREATE TABLE "User" (  
2     id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
3     name VARCHAR(100) NOT NULL,  
4     email VARCHAR(100) NOT NULL UNIQUE,  
5     role VARCHAR(20) NOT NULL CHECK (role IN ('student', 'teacher')),  
6     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
7 );  
8  
9 CREATE TABLE "Course" (  
10    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
11    title VARCHAR(200) NOT NULL,  
12    description TEXT,  
13    teacher_id INT NOT NULL REFERENCES "User"(id) ON DELETE RESTRICT,  
14    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
15 );  
16  
17 CREATE TABLE "Module" (  
18    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
19    title VARCHAR(200) NOT NULL,  
20    order_num INT NOT NULL CHECK (order_num > 0),  
21    course_id INT NOT NULL REFERENCES "Course"(id) ON DELETE CASCADE,  
22    UNIQUE (course_id, order_num)  
23 );  
24  
25 CREATE TABLE "Lesson" (  
26    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
27    title VARCHAR(200) NOT NULL,  
28    type VARCHAR(20) NOT NULL CHECK (type IN ('video', 'article', 'quiz'  
29    ')),  
30    content_url VARCHAR(500),  
31    order_num INT NOT NULL CHECK (order_num > 0),  
32    module_id INT NOT NULL REFERENCES "Module"(id) ON DELETE CASCADE,  
33    UNIQUE (module_id, order_num)  
34 );  
35  
36 CREATE TABLE "Enrollment" (  
37    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
38    student_id INT NOT NULL REFERENCES "User"(id) ON DELETE CASCADE,  
39    course_id INT NOT NULL REFERENCES "Course"(id) ON DELETE CASCADE,  
40    progress DECIMAL(5,2) DEFAULT 0.00 CHECK (progress BETWEEN 0 AND  
41    100),  
42    enrolled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
43    UNIQUE (student_id, course_id)  
44 );  
45  
46 CREATE INDEX idx_course_teacher ON "Course"(teacher_id);  
47 CREATE INDEX idx_module_course ON "Module"(course_id);  
48 CREATE INDEX idx_lesson_module ON "Lesson"(module_id);  
49 CREATE INDEX idx_enrollment_student ON "Enrollment"(student_id);  
50 CREATE INDEX idx_enrollment_course ON "Enrollment"(course_id);
```

Наполнение базы данных тестовыми данными

Пользователи (преподаватели и студенты)

```
1 INSERT INTO "User" (name, email, role) VALUES
2 ('Andrey Rudov', 'andrey.rudov@lms.ru', 'teacher'),
3 ('Veniamin Oprya', 'veniamin.oprya@lms.ru', 'teacher');
4
5 INSERT INTO "User" (name, email, role) VALUES
6 ('Denis Turubanov', 'denis.turubanov@lms.ru', 'student'),
7 ('Yuri Alekseev', 'yuri.alekseev@lms.ru', 'student'),
8 ('Maxim Fialkovsky', 'maxim.fialkovsky@lms.ru', 'student');
```

Курсы

```
1 INSERT INTO "Course" (title, description, teacher_id) VALUES
2 ('Python Course', 'Learning Python: from first programs to confident
3   problem solving.', 1),
4 ('Python and Mathematics', 'Math library and applied problems in Python
5   .', 1),
6 ('Python and Data Structures', 'Lists, dictionaries, sets and
7   algorithms in Python.', 2);
```

Модули для курсов

```
1 INSERT INTO "Module" (title, order_num, course_id) VALUES
2 ('Introduction to Python', 1, 1),
3 ('Functions', 2, 1);
4
5 INSERT INTO "Module" (title, order_num, course_id) VALUES
6 ('Math Library', 1, 2);
7
8 INSERT INTO "Module" (title, order_num, course_id) VALUES
9 ('Lists and Tuples', 1, 3),
10 ('Dictionaries and Sets', 2, 3),
11 ('Algorithms in Python', 3, 3);
```

Уроки

```
1 INSERT INTO "Lesson" (title, type, content_url, order_num, module_id)
2   VALUES
3   ('What is Python?', 'video', 'https://lms.example.com/python_intro', 1,
4     1),
5   ('Installation and Setup', 'article', 'https://lms.example.com/setup',
6     2, 1),
7   ('First Program', 'video', 'https://lms.example.com/hello', 3, 1),
8   ('Quiz: Basics', 'quiz', 'https://lms.example.com/quiz', 4, 1),
9   ('Quiz Review', 'article', 'https://lms.example.com/quiz_review', 5, 1)
10  ;
```

```

7
8 INSERT INTO "Lesson" (title, type, content_url, order_num, module_id)
  VALUES
9 ('Functions', 'video', 'https://lms.example.com/func', 1, 2),
10 ('Lambda Functions', 'video', 'https://lms.example.com/lambda', 2, 2);
11
12 INSERT INTO "Lesson" (title, type, content_url, order_num, module_id)
  VALUES
13 ('Basic Math Functions', 'video', 'https://lms.example.com/math_basic',
    1, 3),
14 ('Trigonometric Functions', 'article', 'https://lms.example.com/trig',
    2, 3),
15 ('Quiz: Math', 'quiz', 'https://lms.example.com/math_quiz', 3, 3);
16
17 INSERT INTO "Lesson" (title, type, content_url, order_num, module_id)
  VALUES
18 ('Lists', 'video', 'https://lms.example.com/list', 1, 4),
19 ('List Methods', 'article', 'https://lms.example.com/list_methods', 2,
    4);
20
21 INSERT INTO "Lesson" (title, type, content_url, order_num, module_id)
  VALUES
22 ('Dictionaries', 'video', 'https://lms.example.com/dicts', 1, 5),
23 ('Dictionary Methods', 'article', 'https://lms.example.com/dict_methods
    ', 2, 5);
24
25 INSERT INTO "Lesson" (title, type, content_url, order_num, module_id)
  VALUES
26 ('Binary Search', 'article', 'https://lms.example.com/binary', 1, 6),
27 ('Algorithm Quiz', 'quiz', 'https://lms.example.com/algo_quiz', 2, 6);

```

Зачисления студентов на курсы

```

1 INSERT INTO "Enrollment" (student_id, course_id, progress) VALUES
2 (3, 1, 0.0),
3 (3, 2, 0.0),
4 (3, 3, 0.0);
5
6 INSERT INTO "Enrollment" (student_id, course_id, progress) VALUES
7 (4, 1, 0.0),
8 (4, 2, 0.0),
9 (4, 3, 0.0);
10
11 INSERT INTO "Enrollment" (student_id, course_id, progress) VALUES
12 (5, 1, 0.0),
13 (5, 2, 0.0),
14 (5, 3, 0.0);

```

Простые DML-операции

Вставка новых записей

```
1 INSERT INTO "Module" (title, order_num, course_id) VALUES
2 ('Statistics', 2, 2);
3
4 INSERT INTO "Lesson" (title, type, content_url, order_num, module_id)
5     VALUES
6 ('Introduction to Statistics', 'video', 'https://lms.example.com/
    stats_intro', 1, 7),
7 ('Quiz: Statistics', 'quiz', 'https://lms.example.com/stats_quiz', 2,
    7);
```

Обновление существующих записей

```
1 UPDATE "Enrollment"
2 SET progress = LEAST(progress + 20.0, 100.0)
3 WHERE student_id = (SELECT id FROM "User" WHERE name = 'Denis Turubanov
4     ')
5     AND course_id = (SELECT id FROM "Course" WHERE title = 'Python Course
6     ');
7
8 UPDATE "Enrollment"
9 SET progress = 100.0
10 WHERE student_id = (SELECT id FROM "User" WHERE name = 'Yuri Alekseev')
11     AND course_id = (SELECT id FROM "Course" WHERE title = 'Python and
12     Mathematics');
13
14 UPDATE "Enrollment"
15 SET progress = LEAST(progress + 5.0, 100.0)
16 WHERE course_id = (SELECT id FROM "Course" WHERE title = 'Python and
17     Data Structures')
18     AND progress < 100.0;
```

Удаление записей

```
1 DELETE FROM "Lesson"
2 WHERE title = 'Quiz Review'
3     AND module_id = (SELECT id FROM "Module" WHERE title = 'Introduction
4     to Python');
5
6 DELETE FROM "Enrollment"
7 WHERE student_id = (SELECT id FROM "User" WHERE name = 'Maxim
8     Fialkovsky')
9     AND course_id = (SELECT id FROM "Course" WHERE title = 'Python and
10     Mathematics');
```

Запросы с агрегацией

Количество студентов на каждом курсе (COUNT + GROUP BY + ORDER BY)

```
1 SELECT c.title AS course_name, COUNT(e.id) AS students_count
2 FROM "Course" c
3 LEFT JOIN "Enrollment" e ON c.id = e.course_id
4 GROUP BY c.id, c.title
5 ORDER BY students_count DESC;
```

Средний прогресс студентов по каждому курсу (AVG + HAVING + ORDER BY)

```
1 SELECT c.title AS course_name, AVG(e.progress) AS avg_progress
2 FROM "Course" c
3 JOIN "Enrollment" e ON c.id = e.course_id
4 GROUP BY c.id, c.title
5 HAVING AVG(e.progress) > 0
6 ORDER BY avg_progress DESC;
```

Максимальный и минимальный прогресс по курсам (MAX + MIN + GROUP BY)

```
1 SELECT c.title AS course_name,
2       MIN(e.progress) AS min_progress,
3       MAX(e.progress) AS max_progress,
4       AVG(e.progress) AS avg_progress
5 FROM "Course" c
6 JOIN "Enrollment" e ON c.id = e.course_id
7 GROUP BY c.id, c.title;
```

Общая статистика по всем зачислениям (SUM, AVG, COUNT, MIN, MAX)

```
1 SELECT
2     COUNT(*) AS total_enrollments,
3     SUM(progress) AS total_progress,
4     AVG(progress) AS avg_progress,
5     MIN(progress) AS min_progress,
6     MAX(progress) AS max_progress
7 FROM "Enrollment";
```

Запросы с соединениями таблиц

Список студентов и их прогресс по курсам (INNER JOIN)

```
1 SELECT u.name AS student_name, c.title AS course_name, e.progress
2 FROM "User" u
3 INNER JOIN "Enrollment" e ON u.id = e.student_id
4 INNER JOIN "Course" c ON e.course_id = c.id
5 WHERE u.role = 'student'
6 ORDER BY u.name, e.progress DESC;
```

Курсы с преподавателями и количеством модулей (LEFT JOIN)

```
1 SELECT c.title AS course_name, t.name AS teacher_name, COUNT(m.id) AS
   modules_count
2 FROM "Course" c
3 LEFT JOIN "User" t ON c.teacher_id = t.id
4 LEFT JOIN "Module" m ON c.id = m.course_id
5 GROUP BY c.id, c.title, t.name
6 ORDER BY modules_count DESC;
```

Полная структура курса (INNER JOIN)

```
1 SELECT c.title AS course, m.title AS module, l.title AS lesson, l.type
2 FROM "Course" c
3 INNER JOIN "Module" m ON c.id = m.course_id
4 INNER JOIN "Lesson" l ON m.id = l.module_id
5 WHERE c.title = 'Python Course'
6 ORDER BY m.order_num, l.order_num;
```

Все курсы и количество записанных студентов (LEFT JOIN + агрегация)

```
1 SELECT c.title AS course_name, COUNT(e.student_id) AS students_enrolled
2 FROM "Course" c
3 LEFT JOIN "Enrollment" e ON c.id = e.course_id
4 GROUP BY c.id, c.title
5 ORDER BY students_enrolled DESC;
```

Студенты и количество курсов, на которые они записаны (INNER JOIN + GROUP BY)

```
1 SELECT u.name AS student_name, COUNT(e.course_id) AS courses_count
2 FROM "User" u
3 INNER JOIN "Enrollment" e ON u.id = e.student_id
4 WHERE u.role = 'student'
5 GROUP BY u.id, u.name
6 ORDER BY courses_count DESC;
```

Создание представлений

Представление "StudentCourseInfo"— информация о студентах и их курсах

```
1 CREATE VIEW StudentCourseInfo AS
2 SELECT u.id AS student_id, u.name AS student_name, u.created_at AS
   registered_date,
3       c.id AS course_id, c.title AS course_name, e.progress, e.
   enrolled_at
4 FROM "User" u
5 JOIN "Enrollment" e ON u.id = e.student_id
6 JOIN "Course" c ON e.course_id = c.id
7 WHERE u.role = 'student';
```

Представление "CourseModuleCount"— количество модулей в каждом курсе

```
1 CREATE VIEW CourseModuleCount AS
2 SELECT c.id AS course_id, c.title AS course_name, c.created_at,
3       COUNT(m.id) AS modules_count
4 FROM "Course" c
5 LEFT JOIN "Module" m ON c.id = m.course_id
6 GROUP BY c.id, c.title, c.created_at;
```

Представление "StudentTotalProgress"— общий прогресс каждого студента

```
1 CREATE VIEW StudentTotalProgress AS
2 SELECT u.id AS student_id, u.name AS student_name, u.created_at,
3       COUNT(e.course_id) AS total_courses,
4       AVG(e.progress) AS avg_progress,
5       MAX(e.enrolled_at) AS last_activity
6 FROM "User" u
7 JOIN "Enrollment" e ON u.id = e.student_id
8 WHERE u.role = 'student'
9 GROUP BY u.id, u.name, u.created_at;
```